

Dell Warranty - TechDirect API (SBIL / EAPI v5)

Purpose:

This procedure retrieves **warranty entitlement information** for Dell devices using **Service Tags**, via the **Dell TechDirect Warranty API v5 (SBIL / EAPI)**.

The output is a CSV file used for **asset disposal validation, audits, and compliance reporting**.

PREREQUISITES

Access and Accounts:

- Dell TechDirect account ([TechDirect Management Portal](#))
- API application Approved for Production ([TechDirect API Management Portal](#))
- Warranty / Asset Entitlement API enabled ([MUST be marked "Approved for Production"](#))

Credentials:

- Client ID
- Client Secret

([Both generated in the TechDirect API Portal](#))

System Requirements:

- Windows workstation
- PowerShell 5.x or newer
- Microsoft Excel installed (script uses Excel COM object)

Input File:

- Excel file (.xlsx)
- Service Tags in column A
- Header row in row 1
- One Service Tag per row
- Dell Service Tags = 7 alphanumeric characters

SCRIPT CONFIGURATION

Only the following values need to be changed/prepared:

```
$ClientID    = "YOUR_CLIENT_ID_HERE"  
$ClientSecret = "YOUR_CLIENT_SECRET_HERE"
```

```
$InputFile = "C:\Path\To\ServiceTags.xlsx"  
$OutputCSV = "C:\Path\To\Dell_Warranty_Results.csv"
```

 **Do not change** the API URLs unless Dell updates their documentation.

HOW THE SCRIPT WORKS

STEP 1 – Load Service Tags:

- Reads column A from the Excel file
- Filters to valid 7-character Dell Service Tags

STEP 2 – Authenticate (OAuth 2.0):

- Uses Client Credentials grant
- Requests token from:
<https://apigtwb2c.us.dell.com/auth/oauth/v2/token>

 **Token validity** is up for 60 minutes only.

STEP 3 – Query Warranty API:

- Endpoint (per Dell Warranty API Spec v2.5, section 3.2.1):
GET <https://apigtwb2c.us.dell.com/PROD/sbil/eapi/v5/asset-entitlements>

STEP 4 – Parse Response:

- API returns an array
- Devices may have multiple entitlements
- Warranty status logic:
 - Active → EndDate ≥ today
 - Expired → EndDate < today

STEP 5 – Export to Path Provided:

Results are written into a CSV file to the path you have previously specified in the script configuration.

OUTPUT FIELDS (CSV FILE)

Column	Description
ServiceTag	Dell Service Tag
Product	Product line / model
ShipDate	Original ship date
WarrantyType	INITIAL / EXTENDED
StartDate	Warranty start
EndDate	Warranty end
Level	Service level description
Status	Active / Expired

HOW TO RUN THE SCRIPT

1. Save script as:
Dell_Warranty_Lookup.ps1
2. Update configuration values
3. Open PowerShell
4. Run:
.\Dell_Warranty_Lookup.ps1
5. Review of the generated CSV

TROUBLESHOOTING (COMMON ISSUES)

X 400 – Bad Request

Cause: Incorrect query parameter

Fix: Must use “servicetags” – Plural is mandatory (per Dell API Spec v2.5)

X 401/403 – Unauthorized

Cause:

- Wrong Client ID / Secret
- Token expired

Fix:

- Regenerate token
- Confirm credentials in TechDirect

X 500 – Internal Server Error

Cause:

- Wrong endpoint
- API entitlement not enabled

Fix:

- Confirm endpoint is:
</PROD/sbil/eapi/v5/asset-entitlements>
- Verify Warranty API access is enabled in TechDirect

X Missing Devices in Output

Cause:

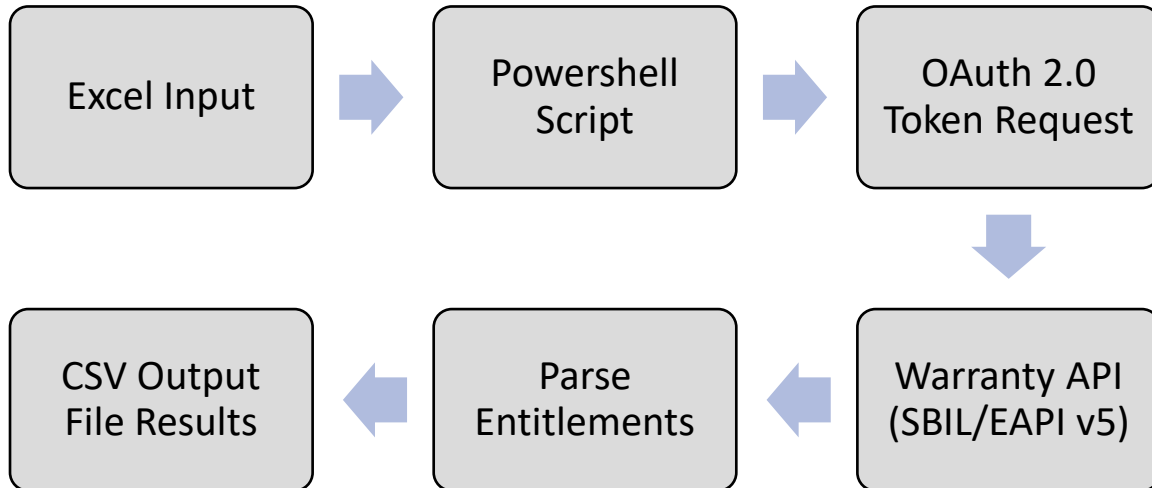
- Non-Dell devices
- Invalid or mistyped Service Tags

Note

The API **does not** return results for:

- Toshiba / HP / Acer / Non-Dell assets
- Invalid/old Service Tags

API FLOW DIAGRAM



DELL DOCUMENTATION REFERENCES

Dell Warranty API Spec – Rev 2.5:

- Section 3.2.1 – Asset Warranty Endpoint
- Section 3.2.2 – Query Parameters (servicetags)
- Sample JSON responses

OAuth TLS 2.0 – Version 1.3:

- Client Credentials grant
- Token endpoint usage
- Bearer token handling

INTENDED USE

- Asset disposal verification
- Warranty audits
- Inventory reconciliation
- Compliance documentation

Next you will find the script in case you want to create the .ps1 file on your own, afterwards a few image examples on how the files and process looks.

FULL SCRIPT (Copy from this point forward)

```
# =====
# Dell Warranty API V5 - Cross-Platform Version
# =====

# ---- CONFIGURATION ----
$ClientID = "CLIENT ID HERE"
$ClientSecret = "CLIENT SECRET HERE"

# Input files
$InputCSV = "INPUT PATH TO \ServiceTags Testing.csv"
$InputXLSX = "C:\Path\To\ServiceTags.xlsx"

# Output file
$OutputCSV = "OUTPUT PATH TO \Dell_Warranty_Results.csv"

# API URLs (DO NOT MODIFY)
$TokenURL = "https://apigtwb2c.us.dell.com/auth/oauth/v2/token"
$BaseURL = "https://apigtwb2c.us.dell.com/PROD/sbil/eapi/v5/asset-entitlements?servicetags="

# =====
# STEP 0 — OS DETECTION (PowerShell 5 SAFE)
# =====
$IsWindows = $false

if ($PSVersionTable.PSEdition -eq "Desktop" -or $env:OS -eq "Windows_NT") {
    $IsWindows = $true
    $OSName = "Windows"
} else {
    $OSName = "Linux/macOS"
}

Write-Host "Detected OS: $OSName" -ForegroundColor Cyan

# =====
# STEP 1 — LOAD SERVICE TAGS
# =====
$Tags = @()

# ---- Try Excel input ONLY on Windows ----
if ($IsWindows -and (Test-Path $InputXLSX)) {

    Write-Host "Using XLSX input (Windows COM mode)..." -ForegroundColor Cyan

    try {
        $Excel = New-Object -ComObject Excel.Application
        $Workbook = $Excel.Workbooks.Open($InputXLSX)
        $Sheet = $Workbook.Sheets.Item(1)

        $row = 2
        while ($Sheet.Cells.Item($row,1).Text -ne "") {

            $tag = $Sheet.Cells.Item($row,1).Text.Trim()

            if ($tag -match '^[A-Za-z0-9]{7}$') {
                $Tags += $tag
            }

            $row++
        }

        $Workbook.Close($false)
```

```

    $Excel.Quit()
}
catch {
    Write-Host "⚠ Excel read failed. Falling back to CSV..." -ForegroundColor Yellow
}
}

# ---- Default: CSV input ----
if ($Tags.Count -eq 0) {

    Write-Host "Using CSV input..." -ForegroundColor Cyan

    try {
        $Tags = Import-Csv $InputCSV | ForEach-Object {
            $_.ServiceTag.Trim()
        } | Where-Object {
            $_ -match '^[A-Za-z0-9]{7}$'
        }
    }
    catch {
        Write-Host "❌ Unable to read CSV file." -ForegroundColor Red
        return
    }
}

Write-Host "Loaded $($Tags.Count) valid Service Tags." -ForegroundColor Green

# =====
# STEP 2 — AUTHENTICATE (OAuth 2.0)
# =====
Write-Host "Requesting OAuth token..." -ForegroundColor Cyan

$pair = "${ClientID}:${ClientSecret}"
$encodedCreds = [System.Convert]::ToBase64String([System.Text.Encoding]::ASCII.GetBytes($pair))

try {
    $tokenResponse = Invoke-RestMethod -Method Post -Uri $TokenURL -Headers @{
        Authorization = "Basic $encodedCreds"
        "Content-Type" = "application/x-www-form-urlencoded"
    } -Body "grant_type=client_credentials"
}
catch {
    Write-Host ("❌ AUTH FAILED -> " + $_.ErrorDetails.Message) -ForegroundColor Red
    return
}

$AccessToken = $tokenResponse.access_token
Write-Host "Token acquired successfully." -ForegroundColor Green

# =====
# STEP 3 — WARRANTY LOOKUPS
# =====
Write-Host "Querying warranty information..." -ForegroundColor Cyan

$Results = @()

foreach ($tag in $Tags) {

    Write-Host ("Checking tag: " + $tag) -ForegroundColor Yellow

    $uri = $BaseURL + $tag

```

```

try {
    $response = Invoke-RestMethod -Method Get -Uri $uri -Headers @{
        Authorization = "Bearer $AccessToken"
        Accept        = "application/json"
    }
}
catch {
    Write-Host ("Error querying tag " + $tag + " -> " + $_.Exception.Message) -ForegroundColor Red
    continue
}

foreach ($asset in $response) {
    foreach ($ent in $asset.entitlements) {

        $Results += [PSCustomObject]@{
            ServiceTag = $asset.serviceTag
            Product    = $asset.productLineDescription
            ShipDate   = $asset.shipDate
            WarrantyType = $ent.entitlementType
            StartDate  = $ent.startDate
            EndDate    = $ent.endDate
            Level      = $ent.serviceLevelDescription
            Status     = if ((Get-Date $ent.endDate) -ge (Get-Date)) { "Active" } else { "Expired" }
        }
    }
}

# =====
# STEP 4 — EXPORT RESULTS
# =====
$Results | Export-Csv -NoTypeInfo -Path $OutputCSV

```

Write-Host "DONE! Warranty results exported to: \$OutputCSV" -ForegroundColor Green

(PASTE AS IS INTO A NOTEPAD AND THEN SAVE AS .PS1, MAKE SURE TO CHANGE ONLY THE HIGHLIGHTED SECTIONS AT THE BEGINNING)

Note in the following image examples that the input file path wasn't set up and the process failed, make sure to double-check the input file itself to avoid these kinds of issues, the name HAS to match as well.

```
Administrator Windows PowerShell
Loading Service Tags from Excel...
Copy, we couldn't find 'C:\ProgramData\Facility\Testing.xlsx'. Is it possible it was moved, renamed or deleted?
At line:24 char:1
+ $Workbook = $Excel.Workbooks.Open($InputFile)
+ ~~~~~
+ CategoryInfo          : OperationStopped: (:) [], COMException
+ FullyQualifiedErrorId : System.Runtime.InteropServices.COMException

You cannot call a method on a null-valued expression.
At line:25 char:1
+ $Sheet = $Workbook.Sheets.Item(1)
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (:) [], RuntimeException
+ FullyQualifiedErrorId : InvokeMethodOnNull

You cannot call a method on a null-valued expression.
At line:26 char:8
+ while ($Sheet.Cells.Item($row,1).Text -ne "") {
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (:) [], RuntimeException
+ FullyQualifiedErrorId : InvokeMethodOnNull

You cannot call a method on a null-valued expression.
At line:28 char:1
+ $Workbook.Close($false)
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (:) [], RuntimeException
+ FullyQualifiedErrorId : InvokeMethodOnNull

Loaded 0 valid service tags.
```

Service Tags Testing.xlsx • Last Modified: March 30							
File Home Insert Draw Page Layout Formulas Data Review View Automate Help							
Clipboard Font Alignment							
L19 A B C D E F G H							
1	Service Tag						
2	6LSRKQ1						
3	HC63HV1						
4	GSCZ5S1						
5	DLWQKV1						
6	GFWM4S1						
7	DDJF7W1						
8	GS9V5S1						
9	ZZD8LQ1						
10	DJF7MQ1						
11	ZZD1LQ1						
12	GSCB6S1						
13	DJDZLQ1						
14	FY3SB42						
15	C3H8082						

Keywords:

Warranty, lookup, Dell, Service, Tags, Tag, .ps1, API, TechDirect, PowerShell, Script, ServiceTag, CSV.

Revisions

Rev.	Page (P) Chapt. (C)	Description	Date Dept./Init.
1	KB-1	Dell Warranty Lookup	05/05/2025 – MX L2/JAOS
2	KB-1	Updated header and first revision of code	14/05/2026 – MTY MX/JAOS
3			
4			